ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 15

Sumário

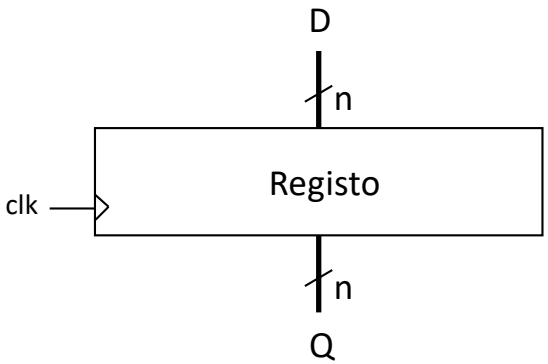
- Registo
- Contadores
- Memórias
- Implementação de funções com memórias

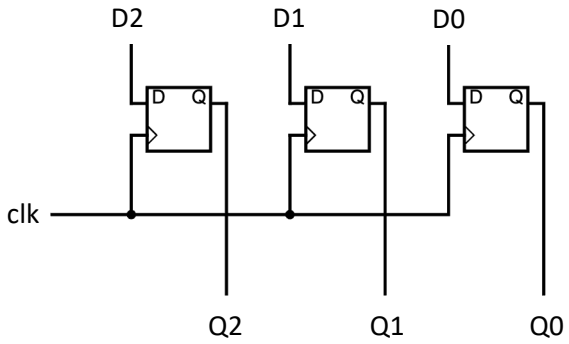
Registo

- Bloco de hardware capaz de memorizar o valor de *n* bits
- É composto por *n* flip-flops
- Pode ter entradas de controlo comuns a todos os FFs (e.g. reset ou set)

Registo

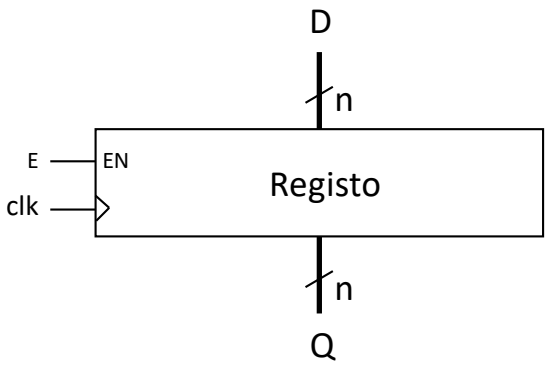
- Bloco de hardware capaz de memorizar o valor de *n* bits
- É composto por *n* flip-flops
- Pode ter entradas de controlo comuns a todos os FFs (e.g. reset ou set)





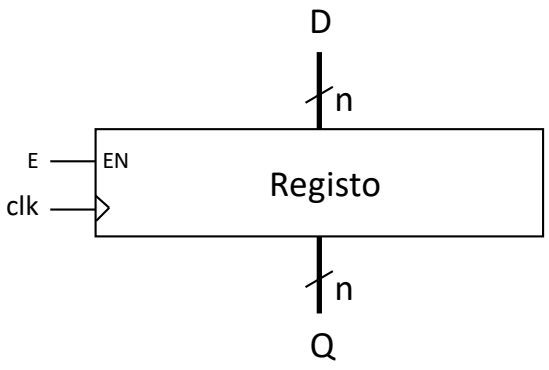
Registo com Enable

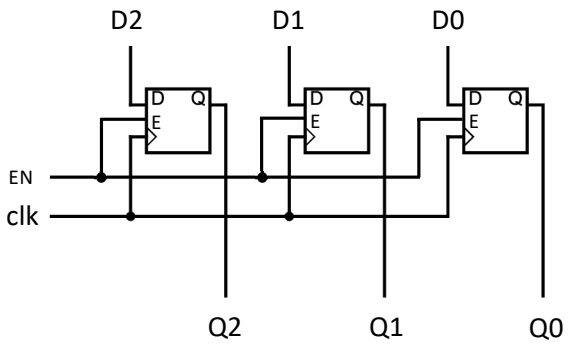
- Enable (EN) – controla quando o registo memoriza um novo valor



Registo com Enable

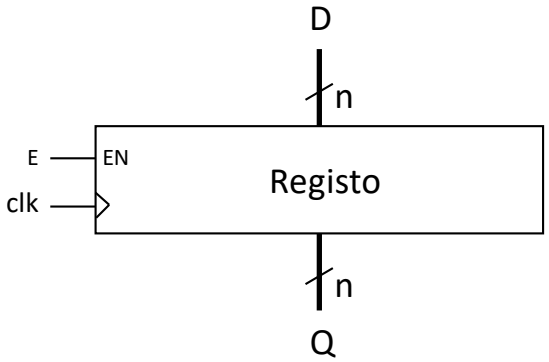
- Enable (EN) – controla quando o registo memoriza um novo valor

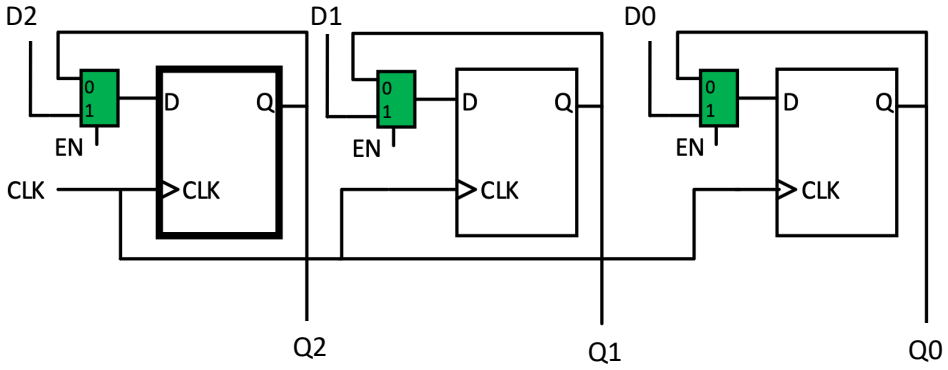




Registo com Enable

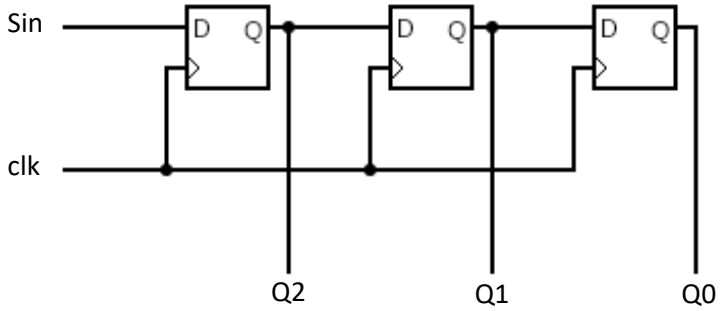
- Enable (EN) – controla quando o registo memoriza um novo valor





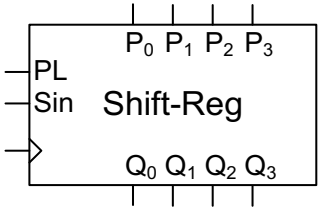
Shift-Register

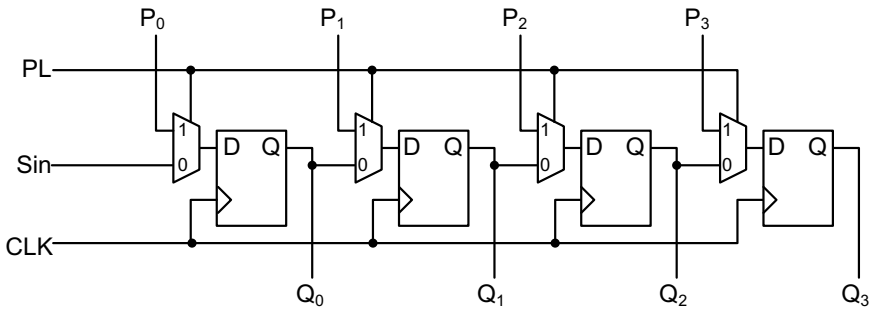
- Módulo *shift-register* (registo de deslocamento)
  - Registo com *n* bits que a cada ciclo de relógio desloca os bits em uma posição
  - Exemplo:
    - deslocamento à direita (*shift-right*)



Shift-Register

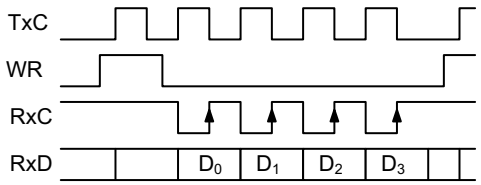
- Módulo *shift-register* (registo de deslocamento)
  - Registo com *n* bits que a cada ciclo de relógio desloca os bits em uma posição
  - Exemplo:
    - permite o carregamento em série ou em paralelo (PL – parallel load)
    - deslocamento à direita (*shift-right*)





Shift-Register

- Exemplo de utilização de um Shift-Register
  - Transmissão dados em série



Contadores

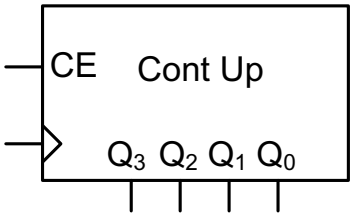
- Um contador é um circuito sequencial que apresenta na sua saída uma sequência de estados que representam uma contagem
- Vamos considerar apenas contadores síncronos, onde a contagem evolui a cada ciclo de relógio
- O módulo do contador é o número de elementos da contagem:
  - Exemplo: contador módulo 8 conta entre 0 e 7
- O contador é uma máquina de estados
  - Pode ser desenhada usando os métodos de síntese de máquinas de estados estudados

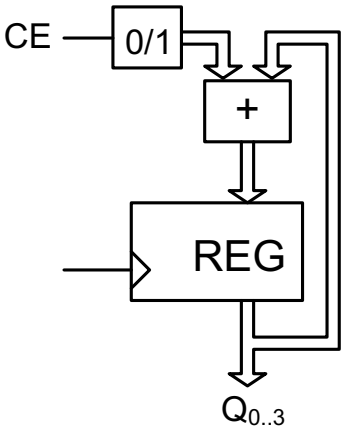
Contadores

- Contador crescente ou decrescente (com ou sem Enable)
- Contador com carregamento paralelo (parallel load)
- Contador multifunções

Contador síncrono crescente

- Contador síncrono crescente (*up counter*)
  - CE – count enable





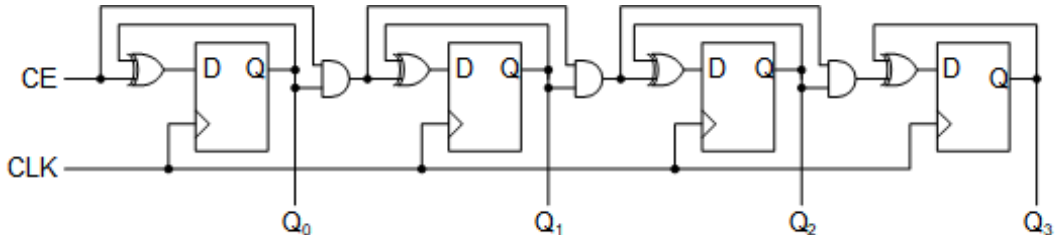
Contador síncrono crescente

- Expressões lógicas
  - $D_0 = Q_0 \oplus CE$
  - $Cy_1 = Q_0 . CE$
  - $D_1 = Cy_1 \oplus Q_1 = CE . Q_0 \oplus Q_1$
  - $Cy_2 = Cy_1 . Q_1 = (CE . Q_0) . Q_1$
  - $D_2 = Cy_2 \oplus Q_2 = (CE . Q_0) . Q_1 \oplus Q_2$
  - $Cy_3 = Cy_2 . Q_2 = ((CE . Q_0) . Q_1) . Q_2$
  - $D_3 = Cy_3 \oplus Q_3 = \left(((CE . Q_0) . Q_1) . Q_2\right) \oplus Q_3 = (CE . Q_0 . Q_1 . Q_2) \oplus Q_3$

$$
\begin{array}{rcccc}
 & Cy_3 & Cy_2 & Cy_1 & \\
 & Q_3 & Q_2 & Q_1 & Q_0 \\
+ & 0 & 0 & 0 & CE \\
\hline
 & D_3 & D_2 & D_1 & D_0
\end{array}
$$

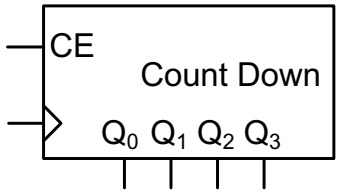
Contador síncrono crescente

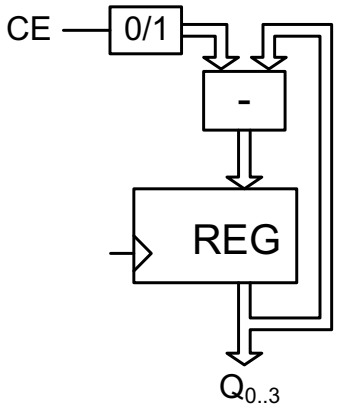
- Possível implementação com flip-flops D



Contador síncrono decrescente

- Contador síncrono decrescente (*down counter*)
  - CE – count enable





Contador síncrono decrescente

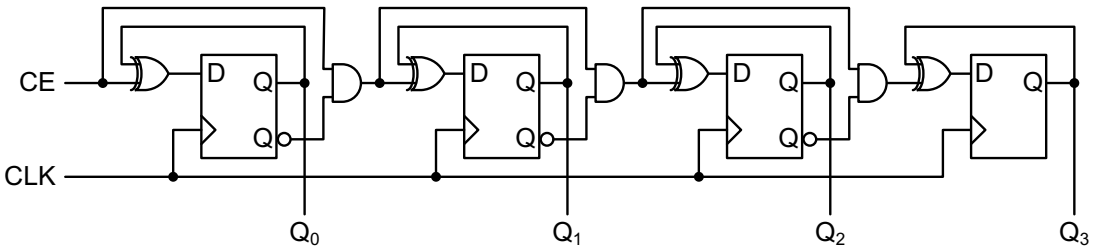
- Expressões lógicas
  - $D_0 = Q_0 \oplus CE$
  - $Bw_1 = CE.\overline{Q}_0$
  - $D_1 = Bw_1 \oplus Q_1 = CE.\overline{Q}_0 \oplus Q_1$
  - $Bw_2 = Bw_1.\overline{Q}_1$
  - $D_2 = Bw_2 \oplus Q_2 = ((CE.\overline{Q}_0).\overline{Q}_1) \oplus Q_2$
  - $Bw_3 = Bw_2.\overline{Q}_2$
  - $D_3 = Bw_3 \oplus Q_3 = (((CE.\overline{Q}_0).\overline{Q}_1).\overline{Q}_2) \oplus Q_3$

$Bw_3 Bw_2 Bw_1$

	Q_3	Q_2	Q_1	Q_0
-	0	0	0	CE
	D_3	D_2	D_1	D_0

Contador síncrono decrescente

- Possível implementação com flip-flops D

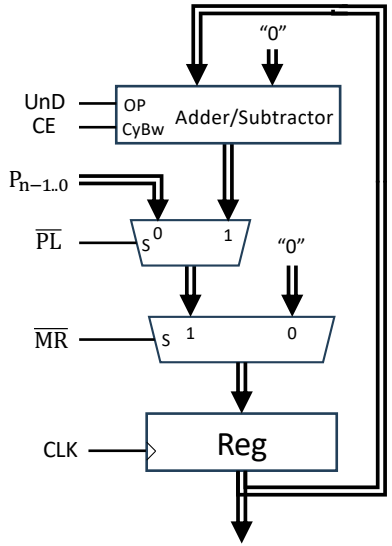


Contador multifunções

- Um contador pode ter algumas entradas adicionais que permitem alterar o seu funcionamento. Por exemplo:
  - CE (count enable) – controla quando a contagem é alterada
  - MR (master reset) – coloca a contagem no valor inicial (tipicamente 0)
  - PL (parallel load) – armazena no contador uma contagem que está presente numa entrada de dados auxiliar
  - UnD (Up/nDown) – controla se o contador faz uma contagem crescente ou decrescente

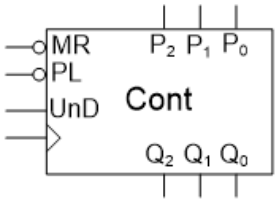
Contador multifunções

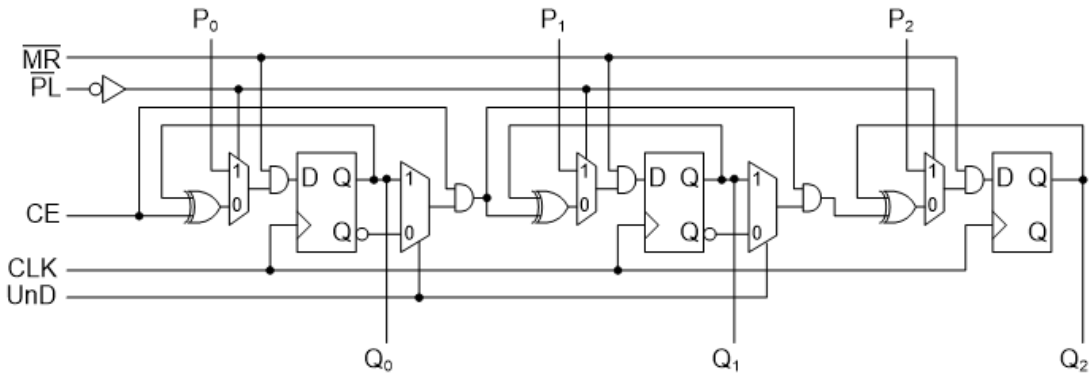
- Exemplo de diagrama de blocos de contador multifunções crescente e decrescente com *Enable*, *Master Reset* síncrono *active_low* e *Parallel Load* síncrono *active_low*



Contador síncrono multifunções

- Exemplo de implementação de um contador multifunções 3-bit com:
  - CE (count enable)
  - MR (master reset) e PL (parallel load) <u>síncronos</u>
  - UnD (Up/nDown) – crescente ou decrescente





Memórias

- As memórias são elementos de circuito com capacidade para armazenar diversas palavras de dados
- O conteúdo de uma memória é acedido através de um endereço, que indica qual a posição da palavra sobre a qual se quer efetuar uma operação (e.g. leitura)
- As memórias podem ser
  - Apenas de leitura – Read Only Memory (ROM)
  - De leitura e escrita
    - De entre estas, a mais comum é a Random Access Memory (RAM)

Memórias

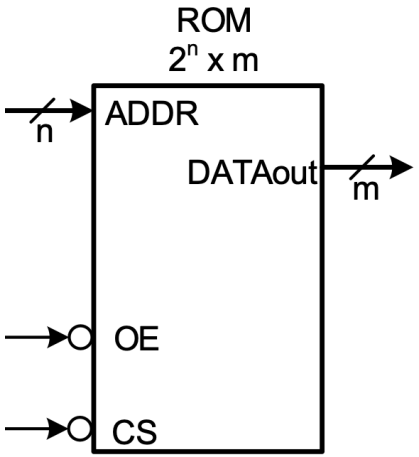
- As memórias são caracterizadas pelo número de palavras que armazenam e pela dimensão de cada palavra
  - Uma memória NxM é capaz de armazenar N palavras de M bits cada
  - A palavra é escolhida através de um endereço => posição
  - Exemplo: uma memória 8x4 contém 8 palavras com 4 bits cada
    - 3 bits de endereço, 4 bits por palavra
    - Para a memória com o conteúdo indicado:
      - endereço 001 (posição $1_{(10)}$) – palavra 1110;
      - endereço 110 (posição $6_{(10)}$) – palavra 1001;

8x4

0	1011
1	1110
2	0101
3	0010
4	1010
5	0001
6	1001
7	0001

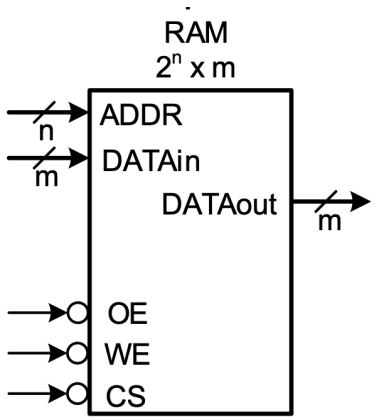
Memória ROM

- **ADDR**: indica o endereço da palavra de onde serão lidos os dados;
- **DATAout**: saída de dados;
- **Output enable** (OE): sinal de controlo que ativa a saída para leitura de dados: quando inativo, a memória coloca as saídas de dados em alta impedância (Z);
- **Chip select** (CS): sinal de controlo que ativa a ROM.



Memória RAM

- **ADDR**: indica o endereço da palavra de onde serão lidos ou para onde serão escritos os dados;
- **DATAin, DATAout**: Entrada (DATAin) de dados para escrita e saída (DATAout) de dados para leitura;
- **Write enable** (WE): sinal de controlo que indica a escrita (quando ativo) ou leitura (inativo) de dados;
- **Output enable** (OE): sinal de controlo que ativa a saída para leitura de dados: quando inativo, a memória coloca as saídas de dados em alta impedância (Z);
- ***Chip select*** (CS): sinal de controlo que ativa a RAM.



Implementação de funções com memória

- Uma memória com capacidade 2^n x m, é capaz de ser utilizada para implementar qualquer função lógica de *n* entradas e *m* saídas
- As entradas são usadas como endereço da memória para indicar qual a palavra de dados a aparecer na saída
- Cada palavra de dados guardada na memória contêm o valor das saídas para a respetiva combinação dos valores das entradas

Implementação de funções com memória

- Exemplo: implementar as seguintes funções lógicas (X e Y) representadas na tabela de verdade com uma ROM:

A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	1
1	1	1	0	1

Implementação de funções com memória

- Exemplo: implementar as seguintes funções lógicas (X e Y) representadas na tabela de verdade com uma ROM:

A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	1
1	1	1	0	1

3 variáveis de entrada (A, B e C):
    3 bits de endereço => 8 (2^3) posições
2 variáveis de saída (X e Y):
    palavras com 2 bits

É necessária uma ROM 8x2

Implementação de funções com memória

- Exemplo: implementar as seguintes funções lógicas (X e Y) representadas na tabela de verdade com uma ROM:

A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	1
1	1	1	0	1

Mapear entradas das funções para bits do endereço:

C -> Addr0
B -> Addr1
A -> Addr2

Mapear saídas das funções para bits da saída de dados:

Y -> DATAout0
X -> DATAout1

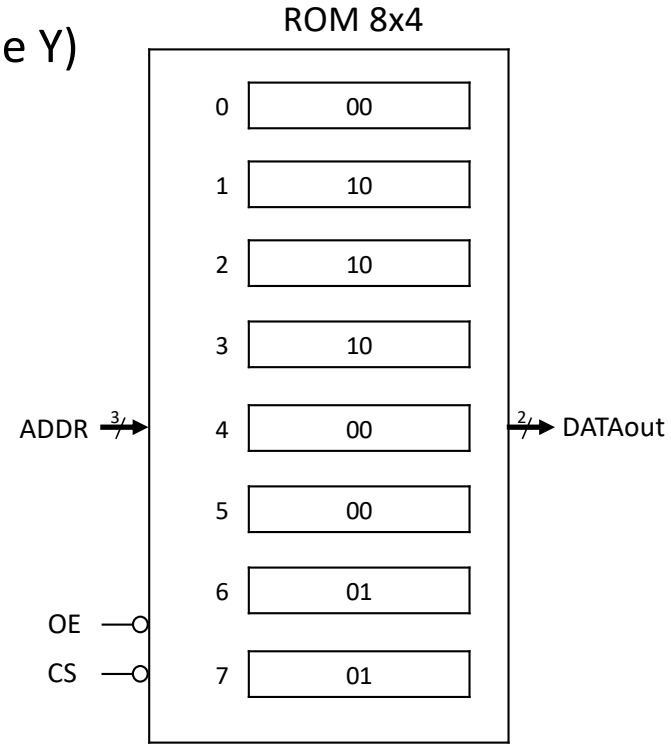
Implementação de funções com memória

- Exemplo: implementar as seguintes funções lógicas (X e Y) representadas na tabela de verdade com uma ROM:

A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	1
1	1	1	0	1

C -> Addr0
B -> Addr1
A -> Addr2

Y -> DATAout0
X -> DATAout1



Temas abordados

Sumário

- Registo
- Contadores
- Memórias
- Implementação de funções com memórias

Próxima aula

- Laboratório D